

9차 연습문항

ROS2 프로그래밍 입문

수강생 공지 사항

- ❖ 제출은 프로그래머스를 통해 해주시기 바랍니다,
- ❖ 파일명은 아래와 같은 형식으로 제출해주세요
 - 교육생번호_이름_교과목_ N차시.pdf
 - ex) DR-11111_홍길동_파이썬_ 1차시.pdf
- ❖ 답은 캡처를 하셔도 되고 텍스트로 넣으셔도 됩니다.
- ❖ 마감 기한은 문제가 나간 주 **일요일 23:59까지**입니다.

1. ROS2에서 python 코드를 이용하여 topic을 publish하기 위하여 사용하는 함수가 무엇인가?

`create_publisher()` : 노드에서 publisher 객체를 생성할 때 사용하는 함수

2. ROS2에서 python 코드를 이용하여 임의의 문자열을 topic으로 publish하는 publisher를 작성한 후 해당 topic을 subscribe하는 subscriber를 작성하시오.

참고: 제공된 코드에서 생략된 부분(...)을 작성하여 코드를 완성하세요.

```
string_publisher.py

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimplePublisher(Node):
    def __init__(self):
        super().__init__('simple_publisher')
        self.publisher_ = ...
        self.timer = self.create_timer(1.0, self.publish_message) # 1초마다 실행
        self.get_logger().info('Publisher node started.')

    def publish_message(self):
        msg = String()
        msg.data = 'Hello, World!!!'
        ...

def main():
    rclpy.init()
    node = SimplePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        print('Publisher node stopped by user.')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

```
string_subscriber.py

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimpleSubscriber(Node):
    def __init__(self):
        super().__init__('simple_subscriber')
        self.subscription = self.create_subscription(...)
        ...

    def listener_callback(self, msg):
        # print msg.data
        self.get_logger().info(f'Received: "{...}"')

def main():
    rclpy.init()
    node = SimpleSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        print('Subscriber node stopped by user.')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

1. string_publisher.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimplePublisher(Node):
    def __init__(self):
        super().__init__('simple_publisher')
        self.publisher_ = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(1.0, self.publish_message)
        self.get_logger().info('Publisher node started.')

    def publish_message(self):
        msg = String()
        msg.data = 'Hello, World!!!'
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: "{msg.data}"')

def main():
    rclpy.init()
    node = SimplePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        print('Publisher node stopped by user.')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

```
2. string_subscriber.py

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimpleSubscriber(Node):
    def __init__(self):
        super().__init__('simple_subscriber')
        self.subscription = self.create_subscription(
            String,
            'chatter',
            self.listener_callback,
            10
        )

    def listener_callback(self, msg):
        self.get_logger().info(f'Received: "{msg.data}"')

def main():
    rclpy.init()
    node = SimpleSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        print('Subscriber node stopped by user.')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```